



Ingenieria Informatica

Procesadores de Lenguaje 25/26

Grupo 81

Practica Final - Tercera Parte

«Traductor de un subconjunto de Lisp a
Forth (back-end)»

121:

Denis Loren Moldovan —

100522240@alumnos.uc3m.es

Jorge Adrian Saghin Dudulea —

100522257@alumnos.uc3m.es

Profesor

Maria Paz

Tabla de contenidos

1. Introducción	2
2. Nuevos tokens	3
3. Variables: setq y setf	4
4. Print y princ	5
5. Defun main y llamada a main	6
6. Bucle: loop while do	7
7. Condicional: if	8
8. Operadores aritméticos y relacionales	9

1. Introducción

Esta tercera parte implementa el back-end del traductor: recibe como entrada un subconjunto de Lisp (generado por el front-end de la práctica anterior) y produce código Forth como salida. La traducción es directa (*Direct Translation*): las acciones semánticas de Bison emiten el código Forth en el mismo momento en que se reduce cada regla gramatical.

2. Nuevos tokens

Se han añadido los siguientes tokens respecto a la versión inicial: OR, NEQ ($\neq$), LTEQ ($\leq$), GTEQ ($\geq$), MOD y NOT. Los operadores de dos caracteres se detectan mediante el mecanismo de *expandable operators*, que lee un segundo carácter al encontrar uno de los caracteres iniciales.

3. Variables: `setq` y `setf`

(`setq` `var` `expr`) declara una variable global en Forth e inicializa su valor. Hay dos variantes: si el valor inicial es un literal numérico emite `variable <nombre> <valor> <nombre> !`; si es una expresión emite `variable <nombre>`, el código de la expresión, y luego `<nombre> !`. (`setf` `var` `expr`) asigna a una variable ya existente, emitiendo solo `<nombre> !` tras la expresión:

```
(setq x 5)           → variable x 5 x !  
(setf x (+ x 1))    → x @ 1 + x !
```

4. Print y princ

- `(print "texto")` emite `.\ " texto\"` cr (con salto de línea).
- `(princ expr)` emite `.` tras evaluar la expresión (sin salto de línea).
- `(princ "texto")` emite `.\ " texto\"` (sin salto de línea).

5. Defun main y llamada a main

La definición de la función principal se traduce a una palabra Forth. La acción semántica emite `: main` al reducir la cabecera y `;` al cerrar el cuerpo. La llamada (`main`) emite simplemente `main`:

```
(defun main () <cuerpo>) → : main <cuerpo> ;
```

6. Bucle: loop while do

El bucle (loop while <cond> do <cuerpo>) sigue el patrón BEGIN ... WHILE ... REPEAT de Forth. Las tres acciones semánticas emiten BEGIN antes de la condición, WHILE tras evaluarla y REPEAT al cerrar:

```
(loop while (< i 10) do (setf i (+ i 1))) → BEGIN i @ 10 < WHILE i @ 1  
+ i ! REPEAT
```


7. Condicional: if

Las acciones emiten IF, ELSE y THEN en los momentos adecuados de la reducción:

```
(if (< x 0) (setf x 0)) → x @ 0 < IF 0 x ! THEN  
(if (< x 0) (setf x 0) (setf x 1)) → x @ 0 < IF 0 x ! ELSE 1 x ! THEN
```

8. Operadores aritméticos y relacionales

Se han añadido las siguientes reglas a la producción `expression`, todas con traducción directa en notación postfija:

Lisp	Forth	Lisp	Forth	Lisp	Forth
<code>(+ a b)</code>	<code>a b +</code>	<code>(and a b)</code>	<code>a b and</code>	<code>(= a b)</code>	<code>a b =</code>
<code>(* a b)</code>	<code>a b *</code>	<code>(or a b)</code>	<code>a b or</code>	<code>(/= a b)</code>	<code>a b = 0=</code>
<code>(/ a b)</code>	<code>a b /</code>	<code>(not a)</code>	<code>a 0=</code>	<code>(< a b)</code>	<code>a b <</code>
<code>(- a b)</code>	<code>a b -</code>	<code>(<= a b)</code>	<code>a b <=</code>	<code>(> a b)</code>	<code>a b ></code>
<code>(- a)</code>	<code>a negate</code>	<code>(mod a b)</code>	<code>a b mod</code>	<code>(>= a b)</code>	<code>a b >=</code>

La desigualdad `/=` se traduce como `= 0=` y not como `0=`, ya que Forth no tiene operadores dedicados para estos casos.